

Correcciones mínimas y regiones críticas

Objetivo

Para este punto se requiere eliminar patrones de espera innecesarios, reducir el tamaño de las regiones críticas y garantizar consistencia del estado compartido, sin afectar la arquitectura general del sistema.

Aplicación

Problema 1

En la clase SnakeRunner, cada serpiente ejecuta un ciclo infinito controlado únicamente por `Thread.sleep` este diseño, no es busy-wait clásico, pero no reacciona inmediatamente a eventos globales como pausa o reanudación del juego lo que obliga a cada hilo despertarse periódicamente aunque el juego esté pausado.

Solución

Introducir un estado de ejecución compartido junto con `wait/notifyAll` para suspender realmente los hilos cuando el juego está en pausa.

Problema 2

El método `step()` está completamente sincronizado esto garantiza seguridad, pero el bloqueo cubre toda la lógica, incluso operaciones que no modifican el estado compartido. Reduce el paralelismo cuando hay muchas serpientes.

Solución

Mantener la sincronización solo donde se accede al estado compartido. Reduciendo la región crítica a las estructuras compartidas.

Problema 3

La clase no es thread-safe se asume implícitamente que cada serpiente es manipulada por un único hilo. Genera condiciones de carrera si otro hilo (ej. renderizado) accede al cuerpo.

Solución

La clase Snake utilizaba una estructura `ArrayDeque`, la cual no es segura en contexto concurrente. Aunque se asumía que cada serpiente era manipulada principalmente por un único hilo, el acceso concurrente desde otros componentes (como la interfaz gráfica) podía generar condiciones de carrera.

La corrección consistió en sincronizar únicamente los métodos que acceden o modifican el cuerpo de la serpiente, protegiendo así la región crítica mínima sin introducir bloqueos globales ni afectar el rendimiento.